

Project Milestone 4

CSCI 335 — Williams College

May 2026

Dean Safran

Project Rules Compliance Statement

We certify that this submission was produced in accordance with all course project rules, including originality requirements and the prohibition on the use of large language models (LLMs) in the creation of the submitted work.

1 Project Description

We built a music streaming database application that allows users to manage playlists, browse songs and artists, and view listening statistics. The application uses an Oracle Autonomous Database backend with a Node.js/Express server and a browser-based frontend. Users can perform full CRUD operations on playlists and users, search songs by artist, filter songs with dynamic conditions, and view aggregated statistics broken down by genre and nationality.

2 Schema Changes from Milestone 2

The following changes were made to the schema submitted in Milestone 2:

- Renamed `Users` to `AppUsers` to avoid conflict with Oracle's reserved keyword `USERS`.
- Extracted `ZipInfo` as a separate table to normalize city/state data that was previously stored directly in `Users`.
- Extracted `SubscriptionFee` as a separate table so that monthly fee data is not duplicated across every user row.
- Replaced the `BelongsTo` relation with `SongGenre` and replaced `Includes` with `AlbumSong` for clearer naming.
- Removed `MemberOf` as it was not needed given our data model.
- Added `nationality` to `Artist`, and added `releaseYear`, `listens`, and `link_to_source` to `Song` to support aggregation queries.

3 SQL Queries

3.1 INSERT — Add Song to Playlist

File: `connectApp.js`, function `addSongToPlaylist`, line 163

Inserts a new row into `InPlayList` after validating that both the playlist and song exist. If the playlist or song is not found, an appropriate error message is returned to the user. The `ORA-00001` error is caught to handle duplicate inserts gracefully.

```
INSERT INTO InPlayList
(name, username, SID, date_time_added, date_time_last_played)
VALUES
(:name, :username, :sid, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)
```

3.2 UPDATE — Update User Profile

File: `connectApp.js`, function `updateUser`, line 238

Updates one or more non-primary-key attributes of `AppUsers` (`subscription_tier`, `zip`, `gender`). The user selects a username from a dynamically loaded list before making changes. The `SET` clause is built dynamically based on which fields the user chose to update, so only provided fields are changed. Note: Oracle does not support `ON UPDATE CASCADE`, so foreign key references to `AppUsers` are not automatically updated.

```
UPDATE AppUsers
SET subscription_tier = :tier, zip = :zip, gender = :gender
WHERE username = :username
```

3.3 DELETE — Delete Playlist (Cascade)

File: connectApp.js, function deletePlaylist, line 409

Deletes a playlist and manually cascades the deletion to all dependent tables: InPlayList, CanAccess, Favorites, Downloaded, and History, before finally deleting from Playlist. All deletes occur in a single transaction.

```
DELETE FROM InPlayList WHERE name = :name AND username = :username;
DELETE FROM CanAccess WHERE name = :name AND ownerUsername = :username;
DELETE FROM Favorites WHERE name = :name AND username = :username;
DELETE FROM Downloaded WHERE name = :name AND username = :username;
DELETE FROM History WHERE name = :name AND username = :username;
DELETE FROM Playlist WHERE name = :name AND username = :username;
```

3.4 Selection — Filter Songs with AND/OR Conditions

File: connectApp.js, function selectSongs, line 546

Allows the user to filter songs using up to three conditions with AND/OR logic. The user selects a field (e.g. Song Title, Artist, Year), an operator (e.g. equals, contains, greater than), and a value. The WHERE clause is built dynamically at runtime from validated inputs. No SQL is directly exposed to the user. Allowed fields and operators are whitelisted server-side.

```
SELECT DISTINCT s.SID, s.songName, a.artistName, s.releaseYear,
               s.length_ms, s.listens, s.popularity, g.genreName
FROM Song s, Sings si, Artist a, SongGenre sg, Genre g
WHERE s.SID = si.SID
      AND si.AID = a.AID
      AND s.SID = sg.SID
      AND sg.genreID = g.genreID
      AND (<user-built conditions with AND/OR>)
```

3.5 Projection — Choose Song Fields to Display

File: connectApp.js, function projectSongs, line 502

The user selects which fields to display (Song Title, Artist, Year, Length, Listens, Popularity, Genre) using checkboxes. Only the selected columns are included in the SELECT clause. Non-selected fields do not appear in the result. The field mapping is whitelisted server-side to prevent SQL injection.

```
SELECT DISTINCT <user-selected fields>
FROM Song s
LEFT JOIN Sings si ON s.SID = si.SID
LEFT JOIN Artist a ON si.AID = a.AID
LEFT JOIN SongGenre sg ON s.SID = sg.SID
LEFT JOIN Genre g ON sg.genreID = g.genreID
ORDER BY s.songName
```

3.6 Join — Songs by Artist

File: connectApp.js, function getSongsByArtist, line 461

Given an artist name input, returns all songs by that artist along with the album each song appears on and the song's genre(s). Joins Artist, Sings, Song, AlbumSong, Album, SongGenre, and Genre. Uses LEFT JOINS so songs without albums or genres are still returned.

```

SELECT DISTINCT a.artistName, s.songName, s.releaseYear, s.listens,
               al.albumName, g.genreName
FROM Artist a
JOIN Sings si ON a.AID = si.AID
JOIN Song s ON si.SID = s.SID
LEFT JOIN AlbumSong als ON s.SID = als.SID
LEFT JOIN Album al ON als.albumName = al.albumName AND als.CID = al.CID
LEFT JOIN SongGenre sg ON s.SID = sg.SID
LEFT JOIN Genre g ON sg.genreID = g.genreID
WHERE LOWER(a.artistName) = LOWER(:artistName)
ORDER BY s.songName, g.genreName

```

3.7 Aggregation with GROUP BY — Artist Listens by Nationality

File: connectApp.js, function getNationalityPopularity, line 284

Groups artists by nationality and computes the total listen hours across all songs by artists of that nationality. Results are ordered by total listen hours descending. Uses **GROUP BY** with **SUM** aggregation across three joined tables.

```

SELECT nationality,
       ROUND(SUM(length_ms * listens)/1000/3600) totalListensHRS
FROM Artist
JOIN Sings USING (AID)
JOIN Song USING (SID)
GROUP BY nationality
ORDER BY totalListensHRS DESC

```

3.8 Aggregation with HAVING — Genre Popularity Threshold

File: connectApp.js, function getGenrePopularity, line 300

For each genre, computes the total number of songs and the average listen count. Includes a **HAVING** clause that filters to only show genres where the song count exceeds a user-specified minimum threshold entered via an input field.

```

SELECT genreName, COUNT(SID) totalSongs, ROUND(AVG(s.listens)) avgListens
FROM Genre g
JOIN SongGenre sg USING(genreID)
JOIN Song s USING(SID)
GROUP BY genreName
HAVING COUNT(SID) > :minSong
ORDER BY avgListens DESC

```

3.9 Nested Aggregation with GROUP BY — Top Genre by Average Listens

File: connectApp.js, function getArtistPopularity, line 384

Finds the most popular artists and their most listened song. The inner query groups by artist to count total listens of all songs and find the most listened song. The outer query joins the table returned with **Song** and **Sings** tables for **artistName** and **songName**. The **RIGHT JOIN** maintains the uniqueness of the most listened song, in case an artist has two songs with the same number of listens.

```

SELECT artistName, totalListens, songName mostListened, ROUND(s2.listens/
totalListens*100, 2) percentListened
FROM (
    SELECT AID, MAX(s1.listens) maxListens, SUM(s1.listens) totalListens

```

```

FROM ARTIST
JOIN SINGS USING(AID)
JOIN SONG s1 USING(SID)
GROUP BY AID
HAVING SUM(s1.listens) >= :minListens
) t
JOIN SINGS USING(AID)
RIGHT JOIN SONG s2
ON t.maxListens = s2.listens
AND SINGS.sid = s2.sid
JOIN ARTIST USING(AID)
ORDER BY totalListens DESC;

```

3.10 Division — Users Who Have Every Song From an Album

File: connectApp.js, function divideUsersByAlbum, line 683

Given an album selected from a dropdown, finds all users who have every song from that album in at least one of their playlists. Uses the double-negation division pattern: returns users for whom there does not exist a song from the album that is not in any of their playlists.

```

SELECT u.username
FROM AppUsers u
WHERE NOT EXISTS (
  SELECT als.SID
  FROM AlbumSong als
  WHERE als.albumName = :albumName
    AND als.CID = :cid
    AND NOT EXISTS (
      SELECT ip.SID
      FROM Playlist p, InPlayList ip
      WHERE p.name = ip.name
        AND p.username = ip.username
        AND p.username = u.username
        AND ip.SID = als.SID
    )
)
ORDER BY u.username

```